

EXERCÍCIO 2 – COLETAR DADOS DO CARTOLA FC

Implementação e Gerenciamento de Bancos de Dados NoSQL

Código no GitHub: github.com/joseniltoncirne/Implementacao-e-Geren-de-banco-de-dados/tree/main/individuais/exercicio02-cartola

O que o script faz

O script `cartola_etl.py` pega as informações do Cartola FC direto do site do Cartola e guarda tudo no MongoDB, no banco `cartola_fc_db`. As informações ficam separadas em 3 coleções:

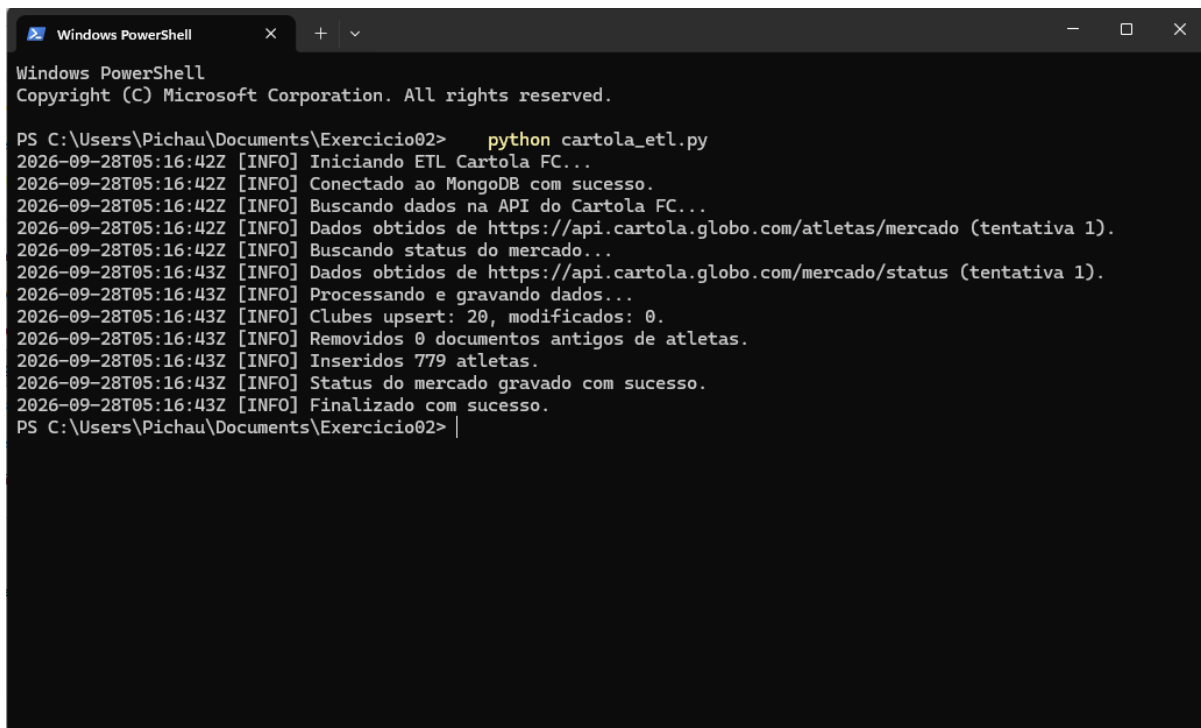
- **clubes_rodada_atual**: os times. Antes de salvar, o script vê se o time já está lá. Se já estiver, só atualiza. Assim nunca fica time repetido.
- **atletas_rodada_atual**: todos os jogadores. Toda vez que roda, o script apaga a lista velha e coloca a nova, anotando o dia e a hora em que pegou as informações.
- **mercado_rodada_atual**: como está o mercado: qual é a rodada, se está aberto ou fechado e quando fecha. Fica guardada só a informação mais nova.

Um detalhe: a parte do site que traz os jogadores não diz qual é a rodada nem se o mercado está aberto. Por isso o script também consulta uma segunda parte do site, que traz essas informações. Sem isso, a coleção do mercado ficaria vazia.

1. Rodando pela primeira vez

```
python cartola_etl.py
```

Na primeira vez, tudo é novo: entram 20 times e 779 jogadores, e as informações do mercado são salvas.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Pichau\Documents\Exercicio02> python cartola_etl.py
2026-09-28T05:16:42Z [INFO] Iniciando ETL Cartola FC...
2026-09-28T05:16:42Z [INFO] Conectado ao MongoDB com sucesso.
2026-09-28T05:16:42Z [INFO] Buscando dados na API do Cartola FC...
2026-09-28T05:16:42Z [INFO] Dados obtidos de https://api.cartola.globo.com/atletas/mercado (tentativa 1).
2026-09-28T05:16:42Z [INFO] Buscando status do mercado...
2026-09-28T05:16:43Z [INFO] Dados obtidos de https://api.cartola.globo.com/mercado/status (tentativa 1).
2026-09-28T05:16:43Z [INFO] Processando e gravando dados...
2026-09-28T05:16:43Z [INFO] Clubes upsert: 20, modificados: 0.
2026-09-28T05:16:43Z [INFO] Removidos 0 documentos antigos de atletas.
2026-09-28T05:16:43Z [INFO] Inseridos 779 atletas.
2026-09-28T05:16:43Z [INFO] Status do mercado gravado com sucesso.
2026-09-28T05:16:43Z [INFO] Finalizado com sucesso.
PS C:\Users\Pichau\Documents\Exercicio02> |
```

2. Rodando de novo

```
python cartola_etl.py
```

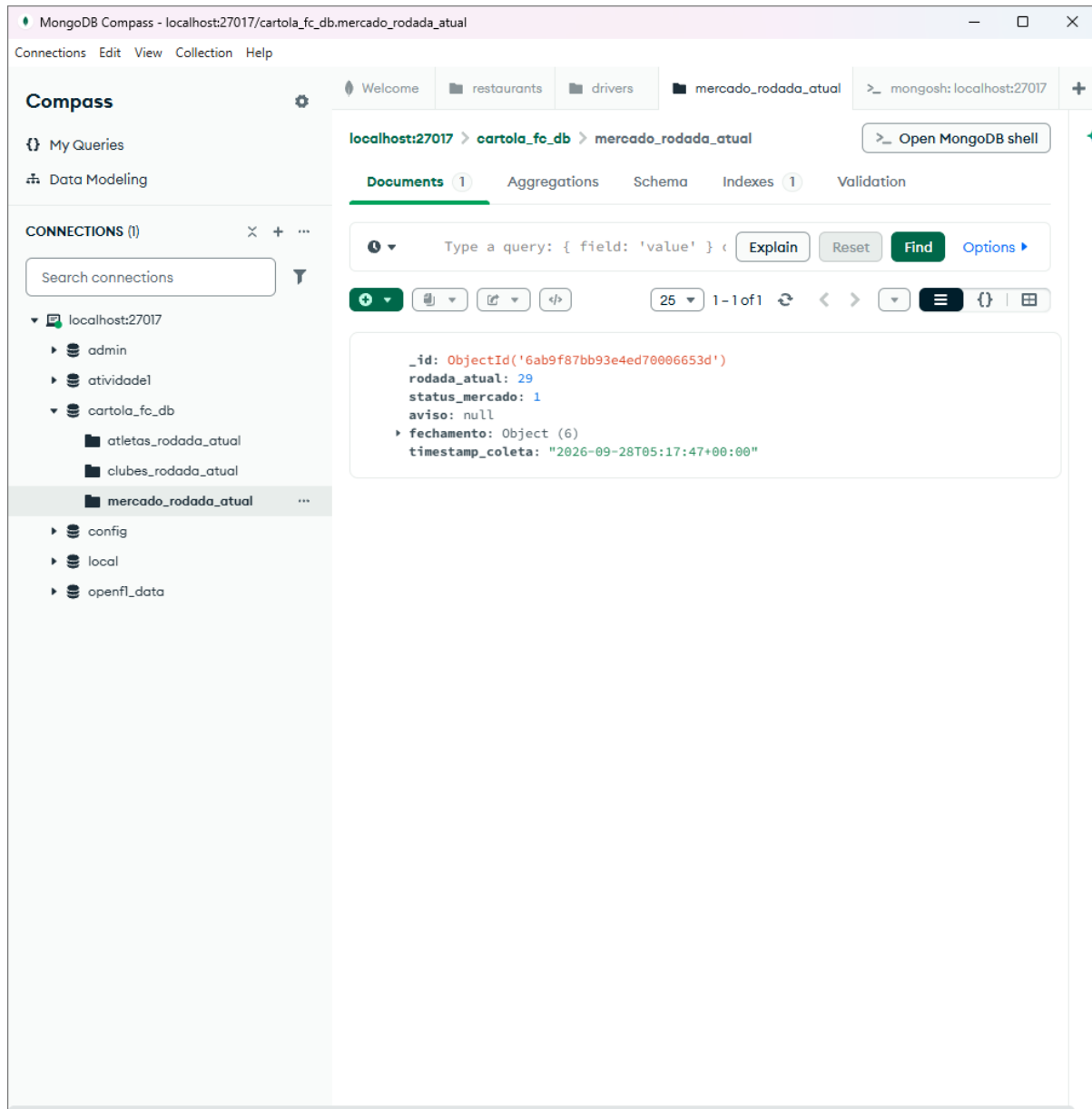
Na segunda vez, nenhum time novo entra, os 20 só são atualizados. Os 779 jogadores antigos são apagados e a lista nova entra no lugar. Ou seja: nada fica repetido.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Pichau\Documents\Exercicio02> python cartola_etl.py
2026-09-28T05:16:42Z [INFO] Iniciando ETL Cartola FC...
2026-09-28T05:16:42Z [INFO] Conectado ao MongoDB com sucesso.
2026-09-28T05:16:42Z [INFO] Buscando dados na API do Cartola FC...
2026-09-28T05:16:42Z [INFO] Dados obtidos de https://api.cartola.globo.com/atletas/mercado (tentativa 1).
2026-09-28T05:16:42Z [INFO] Buscando status do mercado...
2026-09-28T05:16:43Z [INFO] Dados obtidos de https://api.cartola.globo.com/mercado/status (tentativa 1).
2026-09-28T05:16:43Z [INFO] Processando e gravando dados...
2026-09-28T05:16:43Z [INFO] Clubes upsert: 20, modificados: 0.
2026-09-28T05:16:43Z [INFO] Removidos 0 documentos antigos de atletas.
2026-09-28T05:16:43Z [INFO] Inseridos 779 atletas.
2026-09-28T05:16:43Z [INFO] Status do mercado gravado com sucesso.
2026-09-28T05:16:43Z [INFO] Finalizado com sucesso.
PS C:\Users\Pichau\Documents\Exercicio02> python cartola_etl.py
2026-09-28T05:17:46Z [INFO] Iniciando ETL Cartola FC...
2026-09-28T05:17:46Z [INFO] Conectado ao MongoDB com sucesso.
2026-09-28T05:17:46Z [INFO] Buscando dados na API do Cartola FC...
2026-09-28T05:17:46Z [INFO] Dados obtidos de https://api.cartola.globo.com/atletas/mercado (tentativa 1).
2026-09-28T05:17:46Z [INFO] Buscando status do mercado...
2026-09-28T05:17:47Z [INFO] Dados obtidos de https://api.cartola.globo.com/mercado/status (tentativa 1).
2026-09-28T05:17:47Z [INFO] Processando e gravando dados...
2026-09-28T05:17:47Z [INFO] Clubes upsert: 0, modificados: 20.
2026-09-28T05:17:47Z [INFO] Removidos 779 documentos antigos de atletas.
2026-09-28T05:17:47Z [INFO] Inseridos 779 atletas.
2026-09-28T05:17:47Z [INFO] Status do mercado gravado com sucesso.
2026-09-28T05:17:47Z [INFO] Finalizado com sucesso.
PS C:\Users\Pichau\Documents\Exercicio02> |
```

3. Dados salvos no MongoDB

Coleção mercado_rodada_atual: mostra que estamos na rodada 29 e que o mercado está aberto (o número 1 quer dizer aberto).



Coleção clubes_rodada_atual: os 20 times, cada um com o seu número.

The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'CONNECTIONS (1)' section with a search bar and a tree view of the database structure. The main panel shows the 'clubes_rodada_atual' collection with 20 documents. The documents are displayed in a list view, showing fields like _id, abreviacao, escudos, nome, nome_fantasia, and timestamp_coleta.

Connections:

- localhost:27017
 - admin
 - atividade1
 - cartola_fc_db
 - atletas_rodada_atual
 - clubes_rodada_atual**
 - mercado_rodada_atual
 - config
 - local
 - openfl_data

Documents (20):

- _id: 2305**
abreviacao: "MIR"
escudos: Object (3)
nome: "MIR"
nome_fantasia: "MIR"
timestamp_coleta: "2026-09-28T05:17:47+00:00"
- _id: 262**
abreviacao: "FLA"
escudos: Object (3)
nome: "FLA"
nome_fantasia: "FLA"
timestamp_coleta: "2026-09-28T05:17:47+00:00"
- _id: 263**
abreviacao: "BOT"
escudos: Object (3)
nome: "BOT"
nome_fantasia: "BOT"
timestamp_coleta: "2026-09-28T05:17:47+00:00"
- _id: 264**
abreviacao: "COR"
escudos: Object (3)
nome: "COR"
nome_fantasia: "COR"
timestamp_coleta: "2026-09-28T05:17:47+00:00"
- _id: 265**
abreviacao: "BAH"
escudos: Object (3)
nome: "BAH"
nome_fantasia: "BAH"
timestamp_coleta: "2026-09-28T05:17:47+00:00"

Coleção atletas_rodada_atual: os 779 jogadores, cada um com o dia e a hora em que as informações foram pegas.

The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (1)' panel lists the local database 'cartola_fc_db' and its collections, including 'atletas_rodada_atual'. The main panel displays the 'Documents' tab for this collection, showing 779 documents. A query filter is set to '{ field: 'value' }'. Two sample documents are shown:

```
{
  "_id": ObjectId('6ab9f87bb93e4ed700066232'),
  "scout": Object (4)
    jogos_num: 2
    atleta_id: 107203
    rodada_id: 28
    clube_id: 315
    posicao_id: 4
    status_id: 5
    pontos_num: 0
    media_num: 1.85
    variacao_num: 0
    preco_num: 4.39
    entrou_em_campo: false
    slug: "max"
    apelido: "Max"
    apelido_abreviado: "Max"
    nome: "Max Alves da Silva"
    foto: "https://s3.glbimg.com/v1/AUTH_58d78b787ec34892b5aaa0c7a146155f/clubes_202..."
    craque: 0
    timestamp_coleta: "2026-09-28T05:17:47+00:00"
}
```

```
{
  "_id": ObjectId('6ab9f87bb93e4ed700066233'),
  "scout": Object (empty)
    jogos_num: 0
    atleta_id: 146663
    rodada_id: 28
    clube_id: 275
    posicao_id: 1
    status_id: 6
    pontos_num: 0
    media_num: 0
    variacao_num: 0
    preco_num: 1
    entrou_em_campo: false
    slug: "kauan-lima"
    apelido: "Kauan Lima"
    apelido_abreviado: "K. Lima"
    nome: "Kauan Lima da Cunha"
    foto: "https://s3.glbimg.com/v1/AUTH_58d78b787ec34892b5aaa0c7a146155f/clubes_202..."
    craque: 0
}
```

Código

cartola_etl.py

```
import os
import sys
import json
import time
import logging
from datetime import datetime, timezone
from typing import Any, Dict, List, Optional

import requests
from pymongo import MongoClient, UpdateOne
from pymongo.errors import PyMongoError
from dotenv import load_dotenv

LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO").upper()
logging.basicConfig(
    level=LOG_LEVEL,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%dT%H:%M:%SZ",
)
logging.Formatter.converter = time.gmtime

API_BASE_URL = "https://api.cartola.globo.com"
MERCADO_ENDPOINT = f"{API_BASE_URL}/atletas/mercado"
STATUS_MERCADO_ENDPOINT = f"{API_BASE_URL}/mercado/status"

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
MONGO_DB_NAME = os.getenv("MONGO_DB_NAME", "cartola_fc_db")

COL_CLUBES = "clubes_rodada_atual"
COL_ATLETAS = "atletas_rodada_atual"
COL_MERCADO = "mercado_rodada_atual"

def iso_utc_now() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()

def conectar_mongodb() -> MongoClient:
    try:
        client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=8000)
        client.admin.command("ping")
        db = client[MONGO_DB_NAME]
        logging.info("Conectado ao MongoDB com sucesso.")
        return db
    except Exception as e:
        logging.exception("Falha ao conectar no MongoDB: %s", e)
        raise

def buscar_dados_mercado(session: Optional[requests.Session] = None,
                          url: str = MERCADO_ENDPOINT) -> Dict[str, Any]:
    sess = session or requests.Session()
    headers = {
        "Accept": "application/json",
        "User-Agent": "cartola-etl/1.0 (+https://example.internal)",
    }

    max_tentativas = 3
    for tentativa in range(1, max_tentativas + 1):
        try:
            resp = sess.get(url, headers=headers, timeout=30)
            resp.raise_for_status()
            logging.info("Dados obtidos de %s (tentativa %d).", url, tentativa)
            return resp.json()
        except requests.RequestException as e:
            logging.warning("Erro na requisição (tentativa %d/%d): %s", tentativa,
max_tentativas, e)
            if tentativa == max_tentativas:
```

```

        logging.exception("Falha ao buscar %s após %d tentativas.", url,
max_tentativas)
        raise
        time.sleep(2 ** tentativa)

def buscar_status_mercado() -> Dict[str, Any]:
    try:
        return buscar_dados_mercado(url=STATUS_MERCADO_ENDPOINT)
    except requests.RequestException:
        logging.warning("Não foi possível obter /mercado/status; seguindo sem ele.")
        return {}

def processar_e_gravar_dados(db, dados_mercado: Dict[str, Any],
                             status_mercado: Optional[Dict[str, Any]] = None) -> None:
    timestamp = iso_utc_now()

    clubes_obj = dados_mercado.get("clubes", {})
    if isinstance(clubes_obj, dict):
        ops: List[UpdateOne] = []
        for club_id_str, club_data in clubes_obj.items():
            try:
                club_id = int(club_id_str)
            except (TypeError, ValueError):
                club_id = club_data.get("id")
                if club_id is None:
                    logging.warning("Clube com ID inválido será ignorado: %s", club_data)
                    continue

            doc = {
                "_id": club_id,
                "nome": club_data.get("nome"),
                "abreviacao": club_data.get("abreviacao"),
                "escudos": club_data.get("escudos"),
                "nome_fantasia": club_data.get("nome_fantasia"),
                "timestamp_coleta": timestamp,
            }
            ops.append(
                UpdateOne(
                    {"_id": doc["_id"]},
                    {"$set": doc},
                    upsert=True,
                )
            )

        if ops:
            try:
                result = db[COL_CLUBES].bulk_write(ops, ordered=False)
                upserts = (result.upserted_count or 0)
                modified = (result.modified_count or 0)
                logging.info("Clubes upsert: %d, modificados: %d.", upserts, modified)
            except PyMongoError as e:
                logging.exception("Erro ao gravar clubes: %s", e)
                raise
        else:
            logging.warning("Campo 'clubes' não está no formato esperado (dict).")

    atletas_list = dados_mercado.get("atletas", [])
    if not isinstance(atletas_list, list):
        logging.warning("Campo 'atletas' não é uma lista. Valor: %s", type(atletas_list))
        atletas_list = []

    for atleta in atletas_list:
        atleta["timestamp_coleta"] = timestamp

    try:
        delete_res = db[COL_ATLETAS].delete_many({})
        logging.info("Removidos %d documentos antigos de atletas.", delete_res.deleted_count)
        if atletas_list:
            db[COL_ATLETAS].insert_many(atletas_list, ordered=False)
            logging.info("Inseridos %d atletas.", len(atletas_list))
        else:
            logging.info("Nenhum atleta para inserir.")
    except PyMongoError as e:

```

```

        logging.exception("Erro ao gravar atletas: %s", e)
        raise

    fonte = status_mercado or dados_mercado
    mercado_doc = {
        "rodada_atual": fonte.get("rodada_atual"),
        "status_mercado": fonte.get("status_mercado"),
        "aviso": fonte.get("aviso"),
        "fechamento": fonte.get("fechamento"),
        "timestamp_coleta": timestamp,
    }
    if mercado_doc["rodada_atual"] is None:
        logging.warning("Status do mercado veio sem 'rodada_atual' (campos podem ficar
nulos).")

    try:
        db[COL_MERCADO].delete_many({})
        db[COL_MERCADO].insert_one(mercado_doc)
        logging.info("Status do mercado gravado com sucesso.")
    except PyMongoError as e:
        logging.exception("Erro ao gravar status do mercado: %s", e)
        raise

def main() -> int:
    logging.info("Iniciando ETL Cartola FC...")
    try:
        db = conectar_mongodb()
        logging.info("Buscando dados na API do Cartola FC...")
        dados = buscar_dados_mercado()
        logging.info("Buscando status do mercado...")
        status = buscar_status_mercado()
        logging.info("Processando e gravando dados...")
        processar_e_gravar_dados(db, dados, status)
        logging.info("Finalizado com sucesso.")
        return 0
    except Exception as e:
        logging.error("Execução encerrada com erro: %s", e)
        return 1

if __name__ == "__main__":
    sys.exit(main())

```

requirements.txt

```

requests
pymongo
python-dotenv

```